How Tessera works

Two Solana programs, two open protocols, and a website that reads the chain directly.

2.0 · pay later over x402, into the escrow · live on devnet

THE ONE DESIGN DECISION EVERYTHING RESTS ON

The program trusts a balance, *not a message.*

```rust
/// Permissionless on purpose: the only input it trusts is the vault balance.
pub fn confirm_funded(ctx: Context<ConfirmFunded>) -> Result<()> {
    let vault_balance = ctx.accounts.vault.amount;
    let order = &mut ctx.accounts.order;
    require!(order.state == OrderState::AwaitingPayment, TesseraError::InvalidState);
    require!(vault_balance >= order.amount, TesseraError::VaultUnderfunded);
    order.state = OrderState::Funded;
    ...
}
```

A facilitator can refuse a payment but cannot redirect one: the buyer's signature fixes recipient, amount and mint. A dishonest merchant server cannot either: the buyer re-derives the escrow address and reads the order from the chain before signing, and on credit the buy_now it signs carries the price too.

programs/tessera/src/lib.rs. A July 2026 study found rule violations in all 15 x402 facilitators it tested (arXiv 2607.19545), which is why nothing here trusts one.

X402 · A REAL 402 FROM A MERCHANT ON DEVNET, 28 OCT 2026

Three ways to pay one order. *All of them into escrow.*

```
$ curl -i "localhost:4020/agents/atlas/x402/telemetry?buyer=Acy1…t7PW"
  HTTP/1.1 402 Payment Required
  PAYMENT-REQUIRED: eyJ4NDAyVmVyc2lvbiI6MiwicmVzb3VyY2Ui…   (the body below, base64)
  accepts[0]  exact             0.20 USDC  payTo DAwJTW9m…  feePayer: x402.org facilitator
  accepts[1]  exact             0.20 USDC  payTo DAwJTW9m…  feePayer: payai.network facilitator
  accepts[2]  tessera-paylater  0.20 USDC  payTo DAwJTW9m…  buyerLimit 9.297270 · fee 0.004 · 600 s · eligible
  extensions.tessera  order DAwJTW9m… on tESs6b4J… · vault CWAamPv6… · hold 0 s (both Trusted)
                      requestHash 7d4b23dd… · openTx 5WuVmnZV…
```

payTo is the escrow order's own account, so even a plain exact transfer lands in its vault. The pay-later option carries this buyer's live limit, read from the chain when the 402 was made.

NEW IN 2.0 · PAY LATER OVER X402 · PROGRAMS/TESSERA-PAYLATER

One instruction pays the escrow *and confirms it.*

1 · 402
exact first, then the pay-later option with the buyer's limit

2 · pay later
the buyer signs buy_now; the merchant checks, signs and sends it

3 · deliver
the merchant delivers and commits its hash

4 · hold
as long as the two parties' tiers say

5 · release
the escrow pays the merchant, minus its 1%

6 · repay --all
price + 2% to the pool, every loan in one transaction

```rust
pub fn buy_now(ctx: Context<BuyNow>, loan_id: [u8; 32], amount: u64, request_hash: [u8; 32]) -> Result<()> {
    let order = read_order(&a.order)?;    // the escrow's own order: owner, address and type checked
    require_keys_eq!(order.buyer, a.buyer.key(), PayLaterError::NotTheBuyer);
    require!(order.amount == amount, PayLaterError::PriceMismatch);   // the price the buyer signed for
    require!(amount <= limit.saturating_sub(line.outstanding), PayLaterError::CreditLimitExceeded);
    pay_from_pool(&a.pool, ..., a.order_vault.to_account_info(), amount)?;   // pool → the order's vault
    tessera::cpi::confirm_funded(...)?;                                      // the escrow marks it Funded
    ...
}
```

The escrow program is unchanged: buy_now calls its permissionless confirm_funded by CPI, so the buyer can never cancel_unpaid the order for the pool's money.

PAY LATER OVER X402 · A REAL RUN ON DEVNET, ON THE NEW ESCROW TESS6B4J…, 28 OCT 2026

```
$ npm run buy -- --buyer nova --merchant quill --pay-later
  [nova] Quill/summary: 0.25 USDC into escrow Fi68yAxK, hold 10s (merchant Established, me Building, pair trusted)
  [nova] Quill/summary: on credit; limit 1.026626 USDC, owing 0.00; fee 0.005, due in 600s
  settle tx  explorer.solana.com/tx/2mZxEzfu…MnzW8   # signed by the server, Nova and Quill; ConfirmFunded inside
  loan       35R7bZzw…  owe 0.255 USDC (0.25 + 0.005 pool fee) by 07:22:32 UTC
  hash check matches the hash committed on-chain
  final      Released

$ npm run tessera -- repay --all --keypair .keys/agents/nova.json --send
  REPAY ALL  0.255 USDC in one transaction; each loan is closed and its rent comes back to you
  confirmed: 4LUwpKcB…f4jQnY

$ npm run tessera -- credit Acy1…t7PW
  Established 527 · limit 2.865299 USDC · owes 0.00 USDC · record: 3 loans, 3 on time · limit boost +8.74%

$ npm run buy -- --buyer charlie --merchant atlas --pay-later
  NOT BOUGHT not eligible for pay later: the buyer has no credit yet (New, score 40)
```

The limit follows the score: 1.026626 USDC in Nova's 402, as a Building buyer; 2.865299 after this repayment, at Established 527 with three loans repaid on time (boost +8.74%). Charlie, which disputes what it receives, is New. Times in UTC (28 Oct in UTC+8). Lines shortened with ….

ON CHAIN · THE PURCHASE ABOVE, READ BACK FROM DEVNET

```
$ solana confirm -v 2mZxEzfu…MnzW8 --url devnet
  Signature 0  DJ8sWBFJ…  the merchant server, fee payer
  Signature 1  Acy1bCFC…  Nova, the buyer
  Signature 2  2hQkyW5y…  Quill, the order's merchant
  Program LXNPYNYc… invoke [1]                the pay-later program
    Program log: Instruction: BuyNow
    Program Tokenkeg… invoke [2]              pool vault 8.009 → 7.759, order vault 0 → 0.25 USDC
    Program tESs6b4J… invoke [2]              the escrow
      Program log: Instruction: ConfirmFunded
    Program tESs6b4J… success
  Program LXNPYNYc… consumed 112718 compute units, success
  Status: Ok · Finalized
```

explorer.solana.com/tx/2mZxEzfu5ReApNejhCz7GMEwkkBJc7rafjiG3LUiKPNGx1J3oFhxEty3xdLtFfdrJ8Aevauzoy4NX3xVGZJMnzW8?cluster=devnet

FAILED PAYMENTS · README, "FAILED PAYMENTS" · THE REFUNDS AND DISPUTES ARE ON DEVNET

No money left stuck: *it never moves, or it goes back.*

The 402 asks too much

Above the buyer's ceiling, it signs nothing. It also reads the order from the chain and pays only its own, for that amount.

Paid, never delivered

After the deadline anyone may refund: the buyer gets it all, and the merchant's score loses 10%. On devnet: mallory, three orders, three refunds.

A delivery is disputed

The arbiter checks what arrived against the hash committed on chain. On devnet: charlie disputed two valid deliveries; both were resolved for the merchant.

The facilitator fails new

Refused at verify: HTTP 400, nothing sent. Left pending: the merchant watches the vault until it lands (delivered) or can no longer land (400, pay again).

A forged pay-later header new

Checked before the merchant signs, and simulated before anything is sent: HTTP 400, nothing sent, and the buyer may pay another way.

A loan not repaid new

After the grace period anyone marks it in default; the server's keeper does, every 30 s. The line freezes, and the next 402 says so.

Before a buyer even tries, the 402 says `eligible: false` with the reason. If the escrow refunds an order the pool paid for, the refund goes to the buyer and the loan stays owed.

Each case with the file and line that handles it: README, "Failed payments".

A PURCHASE ON THE WEBSITE

Checked in the browser, *then paid into escrow.*

The merchant opens the escrow. The browser checks it on-chain before paying. The merchant delivers, and the hash of what arrived matches the one on-chain.

A new buyer's money is held until it confirms receipt.



A MERCHANT'S CREDIT FILE



Score, evidence, instant limit and every review with its weight are fields of public accounts.

STACK, AND WHAT IS NOT DONE

Programs	Anchor 1.1.2 · escrow (479 KB) + pay-later · devnet
Clients	@solana/kit 8 · Codama-generated
Payments	x402 v2 exact, two facilitators · tessera-paylater, settled by the merchant
Agents	A2A JSON-RPC · a2a-x402 extension v0.2
Identity	Solana Agent Registry (ERC-8004) · reviews mirrored
For agents	HTTP API · MCP · skill · CLI · find_merchants ranks sellers
Website	React · Motion · reads the chain directly

- Devnet only, with test USDC; the demo traffic is our own agents
- Time is compressed: a period is a minute, a loan term 600 s
- Single keys: the upgrade authorities, the arbiter and the pool's authority
- The version 1 escrow, TessSeP5..., lost its keys and is retired; its accounts stay on devnet as history
- An escrow refund goes to the buyer; the loan stays owed
- Unaudited; the A2A endpoint is hand-rolled, not yet run against the official test kit



github.com/edison9733/x402_bnpl